

Transfera project guide

For the team and jury presentation on 9 October 2026

1 Purpose and current scope

Transfera helps a student request a change in academic admission and gives staff a clear way to review it. We have built a working local prototype for two journeys: transferring study to another institution or program, and cancelling an admission. The presentation can show real interactions from submission to a recorded decision.

The problem we are addressing

A transfer involves several questions at once. Who is the student? Which subjects have they completed? Does the destination teach comparable material? Which credits can be recognised, and who has permission to decide? When these details are spread across forms, transcripts and informal messages, the student cannot easily follow the decision and reviewers may lose the reasoning behind it.

Our proposed solution

We bring the application, completed subjects, transcript, comparison, review notes and final decision into one workflow. The student supplies information, the software suggests course matches using text similarity and sample eligibility rules, staff checks the evidence, and an administrator records approval or rejection. Admission cancellation follows a separate request and review process.

What is complete for tomorrow

Account registration and login, administrator-created staff accounts, student profiles, transcript submission, subject comparison, possible duplicate flags, staff review, administrator decisions, cancellation requests and admission status are working. We also provide separate presentation accounts, a Windows setup script, service checks and a manual walkthrough.

How to describe the current product

Present this as a functional academic workflow prototype with fictional records and editable sample rules. It is ready to demonstrate the process. It still needs real institutional policies, verified academic data and production safeguards before an institution uses it with real students. AI is future scope, and ABC ID has been excluded from this version.

2 How a transfer works

The sample journey starts at Sample College A and targets Sample University B. We demonstrate BCA to BCA and BCA to BSc Computer Science, with entry into semester 3. These names and academic records are fictional.

Student submission

The student signs in, completes a profile and chooses the destination program. The request contains a reason, completed subject codes and names, credits, grade points and topics studied. A PDF or text transcript of up to 2 MB is attached. The sample button supplies labelled fictional subjects and a text transcript for a quick demonstration.

Comparison and staff review

Staff compares the request against destination subjects in semesters before the selected entry semester. The query also selects the correct program, institution and curriculum version. The software suggests candidates; staff can inspect both sets of topics and open the transcript. A completed subject can be used only once. Failed grades, insufficient credits and unmatched courses cannot receive credit through the review screen.

Completed subject	Sample evidence	Expected outcome
Intro to Programming	4 credits and grade 8/10	Candidate for Programming Fundamentals
Mathematics	4 credits and grade 3/10	Below the sample pass grade
Data Structures	4 credits and grade 7/10	Eligible after content review
DBMS	2 credits and grade 8/10	Short of the required 4 credits

Staff selects the two eligible matches and saves review notes. The sample has 8 reviewed credits at this point; it has no approved credits yet. An administrator then approves or rejects the request and records a reason and date. An approval awards the reviewed 8 credits. The student can see the same decision after signing in again or refreshing.

For BCA to BCA, two requirements remain in the compared semesters. This is not a claim that only two subjects remain in the entire degree. The prototype catalog covers four subjects in semesters 1 and 2 for each sample program; later-semester curricula are not populated. Program switching may leave different unmatched requirements.

3 Sample rules and duplicate flags

The rules help explain academic review without pretending that one university policy applies everywhere. Faculty can replace the sample values after confirming the intended institution and program.

Rule	Current demonstration value
Destination programs	BCA and BSc Computer Science
Entry and curriculum	Semester 3 and version 2026-demo
Minimum grade	4 out of 10
Completed credits	Must cover the destination subject credits
Title similarity	80 percent suggests a match; 50 to 79 needs closer review
Unmatched subjects	Below 50 percent receives no candidate
Transfer cap	60 credits of a fictional 120-credit degree
Credit decision	Staff reviews; administrator approves or rejects

What a match score means

Names are normalised and selected aliases are recognised, such as DBMS and Database Management Systems. Other titles are compared with TF-IDF cosine similarity, a mathematical text comparison. A score of 100 percent can therefore reflect an exact title or a recognised alias. It does not prove that two syllabi are equivalent. Staff must compare content and supporting evidence.

What a duplicate flag means

A record is flagged when the previous institution and student ID match, or when the name similarity reaches 90 percent and the birth date or phone also matches. A similar name alone is insufficient. Staff sees the matching record and reason; the student sees a general verification notice. The transfer demo intentionally includes similar fictional records. A flag requests a check and does not declare identity fraud.

Where the academic background comes from

Historical MIT ADT Academic Ordinances from 2016 informed the use of academic review, supporting course evidence and a 50 percent transfer cap. This is background for a sample policy, not a statement of current university approval or nationwide compliance. Similarity thresholds, sample degree totals, program choices and entry semester are project decisions that need faculty confirmation.

4 Cancellation and account responsibilities

Cancellation is a separate request about an existing admission. The student provides a reason category, explains the circumstances and acknowledges that approval will cancel that admission.

Review and final decision

While the request is pending or under review, the admission remains active or transferred as it was before. Staff records a recommendation and review notes. An administrator can make the final decision only after review and must give a reason. Approval changes the admission status to cancelled. Rejection keeps the admission unchanged and permits a later request.

Which admission is recorded

The request stores the institution and program at submission. Before a transfer, this is the source admission. After an approved transfer, it is the destination admission. This snapshot helps readers understand what the cancellation decision concerns. Previous request decisions remain in history; cancellation does not erase the transfer record.

Keeping requests consistent

Only one transfer or cancellation may be active for a student. A shared reservation prevents simultaneous submissions from opening conflicting workflows. A cancelled admission cannot submit another transfer or cancellation. Profile editing is locked during an active request and after cancellation. Final decisions cannot be overwritten through the normal API.

Account	Responsibilities	Boundary
Student	Register, maintain profile, submit and track own requests	Cannot review, decide or view others
Staff	Inspect records, compare courses and record review	Cannot make final decisions or appoint staff
Admin	Create staff accounts and record final decisions	Can also review; server checks permissions

The cancellation demo records the application and decision. Refund calculations, financial transactions, institutional clearance, document return and appeal procedures are not implemented. Their rules depend on the institution and should be designed with faculty and administration before being added.

5 Technology and data design

We kept one stack throughout development so teammates can continue without rebuilding the project around new tools. The application runs on one laptop using free local components and does not require a cloud database account or paid AI service.

Part	Technology	Purpose
Interface	React and Vite	Forms, role-aware navigation and request screens
API	Node.js and Express	Validation, access control and workflow actions
Database	MongoDB and Mongoose	Accounts, curricula, requests and decisions
Authentication	JWT and bcrypt	Session tokens and hashed passwords
Matching	Natural library and explicit aliases	Text similarity and candidate suggestions

How the parts connect

The browser uses the API on port 5001. The API connects to a local MongoDB process on port 27018. Vite serves the interface on port 5173. A failed database connection prevents the backend from starting. The health endpoint reports whether the API has a database connection. Each teammate has an independent database on their own laptop; GitHub shares the code and seeded examples, not live application data.

What we store

User records hold account identity, role and a password hash. Student records link a profile to its account. Subjects hold the sample destination curriculum. Transfer records contain the submitted subject snapshot, transcript, comparison, review and final decision. Cancellation records contain the admission snapshot, student reason, review and decision. Admission status is derived from request decisions instead of maintaining a second independent decision state.

Safeguards and practical limits

The backend checks ownership and roles, filters public registration to student accounts, excludes password hashes from responses and restricts transcript access. Unique active-request indexes and version checks protect against duplicate submissions and competing decisions. This is still a local prototype: production use needs secure file storage and scanning, backup and recovery, session hardening and an administrative procedure for a reservation interrupted by a crash. Older mapping APIs remain in the repository; current transfer records are authoritative.

6 Team setup and presentation plan

Use main from the GitHub repository. The short TEAMMATE_SETUP guide is designed for 64-bit Windows laptops using any chosen project folder, including machines without Node or MongoDB installed.

What setup does

Setup downloads the pinned portable runtimes from official sources when missing and verifies their SHA-256 hashes. It installs locked dependencies, creates a random local secret and prepares local data folders. Downloads, caches and application logs stay within the chosen project folder. Initial internet and disk space are required; the MongoDB archive is about 800 MB. Subsequent demonstrations can run locally without an internet connection.

Four accounts for a clear demonstration

Aarav Sharma is the transfer student and Riya Patel is the cancellation student. Both begin with complete fictional profiles and no request history in a fresh database. Staff and admin accounts demonstrate the review and decision roles. All presentation accounts use the local demo password TransferaDemo123!. Seeding adds the accounts and preserves existing requests, so repeating setup does not reset a completed demonstration.

Purpose	Account
Transfer student	transfer@transfera.demo
Cancellation student	cancellation@transfera.demo
Academic review	staff@transfera.demo
Final decision	admin@transfera.demo

For a short pitch, begin with the student problem, show the transfer request and transcript, then switch to staff comparison and administrator approval. Explain the grade failure, credit shortfall and possible duplicate flag. Finish with the separate cancellation student and show the admission status changing only after the final decision. Keep the detailed policy explanation for questions.

Automated verification covers 22 API tests, including ownership, role restrictions, duplicate submissions, both transfer types, failed grades, credit limits, cancellation outcomes and competing decisions. Frontend lint and build pass, and both transfer journeys and cancellation outcomes have been exercised in a browser. The Windows clean-setup trial and service check validate local installation. The team must still run the setup check and manual walkthrough on each teammate laptop before treating it as confirmed there.

7 Future scope and jury discussion

Our next work should strengthen the verified workflow before adding more automation. The final jury version should use rules and records that an institution can stand behind.

First agree the academic policy

Confirm eligibility, permitted transfer stages, grading conversion, required evidence, credit caps and approval authority with faculty. Populate real curricula with versioned subjects, learning outcomes, contact hours and assessment details. Extend matching beyond the four-subject samples and explain requirements across the whole degree.

Then strengthen verification and operations

Introduce institution-confirmed transcripts and a documented process for resolving duplicate records. Add appropriate clearance, refunds and appeals only after the institution defines them. Provide secure uploads, audit history, backups, recovery for interrupted operations, account management and notifications. Test multiple users, devices, unusual academic records and repeat admission changes.

AI assistance after the foundation

AI could help extract transcript fields, compare syllabus topics and draft explanations of possible matches. We would need evaluation examples, error measurement and staff review before using those suggestions. AI should assist academic judgment; approval still belongs to authorised people. This capability is not present in the current demonstration.

Keeping the work affordable

The present laptop demonstration needs no hosting subscription or AI API payment. Continue with deterministic matching and local data while validating the workflow. Shared deployment is a later decision involving secure hosting, database access, privacy, availability and possible costs; we should not promise an unlimited free production service.

How we answer the jury honestly

If asked whether matching is AI, explain that this version uses title aliases and text similarity. If asked whether a flag proves fraud, explain the manual verification step. If asked about university compliance, show the editable sample rules and the need for faculty approval. If asked what is complete, show the working student-to-admin journeys. If asked what remains, describe trusted academic data, institutional procedures and production readiness.

Project repository: <https://github.com/jahnavitongia/Transfera>

Academic background: MIT ADT Academic Ordinances 2016,
https://mituniversity.ac.in/assets_web/pdf/about/Academic_Ordinances_2016.pdf

Runtime sources: Node.js downloads at <https://nodejs.org/en/download> and MongoDB Community installation at <https://www.mongodb.com/docs/manual/tutorial/install-mongodb-on-windows/>